

CONTENTS

- Overview
- 1 The problem workload orchestration solves
- 2 Defining workload orchestration
- 3 Why not just use Kubernetes?
- 4 AI/ML workloads fragment across four tools
- 5 AI/ML workloads need flexible lifecycles
- 6 One scheduler, three first-class job types
- 7 From fragmented ops to unified ops
- 8 Staying ready for what's next
- Glossary
- Check yourself
- Where to go next

Flexible Orchestration for AI & ML: Beyond Kubernetes Automation

Understand why AI and ML workloads outgrow a single Kubernetes cluster, and how a flexible workload orchestrator unifies services, batch jobs, and training runs under one scheduler.

For developers and DevOps engineers deploying AI/ML workloads who want to understand orchestration options beyond Kubernetes · 27 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- Basic familiarity with containers and what a container orchestrator like Kubernetes does
- Basic exposure to YAML or other declarative configuration formats

Overview

A workload orchestrator is the layer of software that decides where an application runs, keeps it running, and repairs it when something fails. Instead of an engineer logging into each server to install, configure, and troubleshoot software one machine at a time, the engineer describes what the application needs — resources, runtime, replica count — and the orchestrator handles placement, scaling, and recovery on its own.

Kubernetes is the best-known example of this idea, and it excels at one specific shape of workload: long-running containerized services that should be kept alive indefinitely. AI and ML systems don't fit that shape cleanly. A training run executes once and exits. An inference service must stay up around the clock. A data pipeline runs on a schedule. Forced onto Kubernetes primitives built for always-on services, these workloads become awkward, and organizations often respond by giving each team its own specialized tool — Kubernetes for web apps, Slurm for GPU training, Airflow for pipelines, containers or manual scripts for everything else. That produces four teams, four tool sets, and four different failure modes to debug.

Flexible orchestration solves this by treating services, batch jobs, and system jobs as equally first-class citizens of one scheduler, described through a single declarative job specification. The same cluster can host a web app, a one-off training run, and a 24/7 inference endpoint, with placement rules (which datacenter, which rack) declared directly in the job file. The result is one tool and one shared body of operational knowledge instead of several fragmented ones — and when the next shift in AI techniques arrives, the response is a new job spec, not a rebuilt platform.

KEY TAKEAWAYS

- ✓ A workload orchestrator automates deployment, scaling, health monitoring, and failure recovery from a declarative description, replacing manual server-by-server administration.
- ✓ Kubernetes was designed to keep long-running containerized services alive; that design assumption makes it awkward for ephemeral, run-once, or scheduled AI/ML workloads.
- ✓ AI/ML organizations commonly end up with four teams on four different tools — Kubernetes for web apps, Slurm for GPU training, Airflow for pipelines, and manual scripts or containers for ML services — which fragments troubleshooting expertise.
- ✓ A flexible orchestrator treats service jobs, batch jobs, and system jobs as first-class scheduler types, so one platform can run all of them instead of forcing every workload through the same primitive.
- ✓ A single declarative job spec bundles resource requirements, replica count, deployment strategy, and placement constraints, replacing the multiple Kubernetes manifests (Deployment, ConfigMap, Secret, storage) a single app usually needs.
- ✓ Consolidating onto one orchestrator turns fragmented ops and siloed per-tool knowledge into unified ops and shared knowledge, cutting a data scientist's turnaround from a multi-day ticket process to a few minutes of self-service.
- ✓ Because AI capabilities change unpredictably — transformer models and GPT-scale inference did not exist a few years ago — the orchestration layer needs to absorb new workload types without requiring the infrastructure itself to be rebuilt.

01 The problem workload orchestration solves

0:00

Deploying the same application across many servers by hand means repeating the same manual login-and-install steps on every machine, and repeating them again for every fix.

Before any orchestration layer exists, deploying an application to a fleet of servers is a manual, per-machine process: log into a server, install and configure the application, and if something breaks, log back in and troubleshoot it directly on that machine. This work does not get easier as it scales — three servers means doing the same steps three times, and a failure on any one of them means a human has to notice it and intervene by hand.

A workload orchestrator removes the per-machine part of this loop. Instead of describing how to install the application on each server, an engineer describes what the application needs: how many resources it requires, what its runtime looks like, and how many copies should be running. The orchestrator takes that description and decides where to place the workload, how to scale it, and how to keep it healthy — all without a human repeating the same manual steps on every box.

KEY POINTS

- Manual deployment requires logging into every server individually to install and troubleshoot software.
- The manual approach doesn't scale: the same steps must be repeated per server, and repeated again for every fix.
- A workload orchestrator instead takes a declarative description of an application's resource and runtime needs.
- The orchestrator, not the engineer, decides placement, scaling, and resiliency from that description.

Manual deployment across three servers

Deploying the same web application to three VMs by hand means SSHing into each one, shipping the release, and restarting the service — and doing it again on every VM if the release changes or something fails. The work triples with the server count instead of staying constant.

```
# manual deploy, repeated per server
for host in vm1 vm2 vm3; do
    ssh admin@$host "sudo systemctl stop myapp || true"
    scp myapp-release.tar.gz admin@$host:/opt/myapp/
    ssh admin@$host "cd /opt/myapp && tar xzf myapp-release.tar.gz && sudo systemctl start myapp"
done
```

02 Defining workload orchestration

1:25

A workload orchestrator runs many kinds of work — web apps, AI/ML jobs, batch jobs — across many machines while automating scheduling, placement, and health monitoring.

Workload orchestration is the general process that lets an organization run many different kinds of applications — web apps, AI and ML workflows, scheduled batch jobs — across multiple servers and environments without manually operating each one. The orchestrator automates three things an operator would otherwise do by hand: scheduling (deciding when a job runs), placement (deciding which machine it runs on), and health monitoring (deciding whether it's still working).

The practical payoff shows up when something fails. If a server running a job goes down, the orchestrator notices that the job's actual state no longer matches its declared desired state, and it automatically reschedules the work elsewhere to restore that state. No one has to be paged, log in, or manually restart anything. A server failure that used to be a crisis requiring immediate human intervention becomes routine, self-correcting behavior instead.

KEY POINTS

- Workload orchestration covers heterogeneous workload types: web apps, AI/ML jobs, and batch jobs, not just one category of application.
- It automates scheduling (when), placement (where), and health monitoring (is it still healthy) so people don't have to.
- Self-healing means the orchestrator detects that actual state no longer matches desired state and corrects it automatically.
- A failed server stops being an emergency that requires a human to notice and intervene.

A desired-state job spec

A job spec that declares '3 replicas should be running' lets the orchestrator continuously compare that desired count against reality. If a server dies and the running count drops to 2, the orchestrator starts a replacement automatically — nobody has to notice the failure or log in to fix it.

```
job "web" {  
  group "app" {  
    count = 3    # desired replica count  
  
    task "server" {  
      driver = "docker"  
      config {  
        image = "myorg/web:1.4.0"  
      }  
    }  
  }  
}
```

03 Why not just use Kubernetes?

2:54

A single Kubernetes application typically needs several separate manifests — Deployment, ConfigMap, Secret, storage — while a workload orchestrator can describe the same app in one declarative job file.

Kubernetes is a capable, widely used orchestrator, but a single deployment rarely stands alone. A realistic Kubernetes application needs a Deployment object plus supporting objects: a ConfigMap for configuration, a Secret for credentials, and a storage definition for persistent data. That's easily four separate YAML files to describe one application, and each one has to stay consistent with the others as the app changes.

A workload orchestrator built around a single declarative job file collapses that into one artifact: replica count, deployment strategy, resource needs, and runtime configuration all live together. An engineer pushes one job file, the orchestrator parses it, determines how many replicas are needed, and deploys them according to the declared strategy — all running on top of a single cluster with an operating system underneath and the orchestrator layer on top. Because there's one file format to learn rather than several object types that must relate to each other correctly, the learning curve is shallower: features can be picked up one at a time, at whatever pace the team needs, rather than requiring a full grasp of the Kubernetes object model up front.

KEY POINTS

- A single Kubernetes app commonly needs four or more separate manifests: Deployment, ConfigMap, Secret, and storage.
- A workload orchestrator job file can bundle replica count, resources, and deployment strategy into one artifact.
- Fewer files means fewer places for a mismatch between config pieces to hide.
- Learning one job-file format is a gentler ramp than learning how several Kubernetes object types interrelate.

One declarative job file for a web app

This single HCL (HashiCorp Configuration Language) job spec declares the container image, replica count, resource requirements, and a canary update strategy for a web app — the equivalent of what a Deployment, ConfigMap, and networking config would otherwise split across several Kubernetes YAML files.

```
job "web-app" {
  datacenters = ["dc1"]

  group "web" {
    count = 3

    update {
      max_parallel = 1
      canary       = 1
    }

    task "server" {
      driver = "docker"
      config {
        image = "myorg/web-app:2.3.0"
        ports = ["http"]
      }
      resources {
        cpu    = 500
        memory = 256
      }
    }
  }

  network {
    port "http" { to = 8080 }
  }
}
```

Organizations often split Kubernetes clusters by workload type, or cram everything into one cluster with heavy namespace and quota management — and different AI/ML teams end up on entirely different tools regardless.

There are two common ways companies try to run AI/ML on Kubernetes, and both create friction. Some run separate clusters for web apps, training, and batch work, which means separate operations teams, separate monitoring, and separate expertise for each cluster. Others push everything onto one shared cluster, which avoids running multiple control planes but requires complex namespace boundaries and resource quotas to keep teams from consuming each other's capacity. Either way, the coordination cost doesn't disappear — it just moves from 'managing multiple clusters' to 'policing one crowded cluster.'

On top of that, a predictable pattern emerges by team. Web teams deploy microservices on Kubernetes. Data scientists often default to Slurm — an HPC job scheduler common in research computing — because it's what they used with GPU jobs in grad school. Data engineers manage pipelines with Airflow. ML teams deploy services in containers, or sometimes just SSH into a box and run custom scripts. That's four teams, four tools, and four distinct sets of expertise required to operate them. When something breaks, there's no shared starting point for diagnosing it — troubleshooting time scales with the number of tools involved, not the number of failures. In a concrete case, a data scientist who wants to run a training job may have to file a ticket with DevOps, wait for a cluster to be provisioned, wait for approval, and get results days later — while the web team, using the same Kubernetes setup for the same organization, deploys continuously with no such wait.

KEY POINTS

- Running separate Kubernetes clusters per workload type multiplies the number of ops and monitoring teams needed.
- Cramming every workload type into one cluster instead requires complex namespace and resource-quota management.
- Data scientists often default to Slurm, an HPC job scheduler, for GPU training jobs.
- Data engineers typically use Airflow for pipeline scheduling; ML teams often deploy via containers or manual scripts.
- Four teams on four tools means troubleshooting requires four different sets of specialized expertise.
- A data scientist's training job can take days to start (ticket, cluster provisioning, approval) while the web team deploys continuously on the same underlying infrastructure.

Same org, two very different turnaround times

In one organization, the web team merges code and it's deployed to Kubernetes the same day through an existing pipeline. A data scientist in the same organization who wants to train a model files a ticket, waits for a cluster to be provisioned, waits for approval, and gets to train the model days later — not because the work is harder, but because it runs through a completely different, less automated tool chain.

05 AI/ML workloads need flexible lifecycles

7:16

Training jobs are ephemeral and run once, inference services must stay up continuously, and pipelines run on a schedule — three different lifecycles that don't map cleanly onto a platform built for one shape of workload.

Kubernetes' core object, the Deployment, keeps a fixed number of pods running indefinitely and restarts them if they stop — exactly right for a web service that should always be up. AI and ML introduce workloads that don't share that assumption. A training job might run for three hours, consume a lot of GPU resources, and then never run again. An inference service, on the other hand, has to be available 24/7, continuously tied to GPU resources. A data pipeline runs on a recurring schedule and otherwise sits idle.

Kubernetes does have primitives for some of this — Jobs and CronJobs exist for batch-style work — but they still feel like an addition to a platform whose primary design center is long-running services, not the primary interface. That mismatch is what makes ephemeral or scheduled AI/ML work awkward on Kubernetes: the tool works, but every batch-shaped workload has to be translated into a shape the platform wasn't originally built around.

KEY POINTS

- Kubernetes Deployments are built to keep a fixed number of long-running pods alive indefinitely.
- Training jobs are ephemeral: they run once for a bounded time and may never run again.
- Inference services need continuous 24/7 availability and a persistent tie to GPU resources.
- Pipelines run on a recurring schedule rather than continuously or once.
- Kubernetes Jobs and CronJobs support batch work, but as an addition to a platform centered on always-on services.

Wedging a training run into a Kubernetes Job

This is how a one-off, GPU-bound training run gets expressed as a Kubernetes object: a separate primitive (Job, not Deployment) with `restartPolicy` set to `Never` so it doesn't get treated like a service that should be kept alive. It works, but it's a different object with different semantics from the Deployments the same cluster uses for the web app — one more thing an operator has to know how to reason about.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: train-model
spec:
  template:
    spec:
      containers:
      - name: trainer
        image: myorg/trainer:latest
        resources:
          limits:
            nvidia.com/gpu: 4
      restartPolicy: Never
    backoffLimit: 2
```

06 One scheduler, three first-class job types

8:45

A flexible orchestrator treats batch jobs, services, and system jobs as equally first-class citizens of one scheduler, with each job's resource and placement needs declared directly in its spec.

In a flexible orchestrator, batch jobs (run once, then exit), service jobs (kept running continuously), and system jobs (run once per eligible node) are all first-class scheduler types, not special cases bolted onto a platform designed for something else. One cluster can host a web app, a training run, and an inference service side by side, each declared with the lifecycle that actually matches how it behaves.

Placement is handled the same declarative way. Each job spec can state exactly where it's allowed to run — spread across one datacenter, spread across a combination of datacenters, or pinned to a specific rack in a specific datacenter. Because that constraint lives in the job spec itself, the scheduler enforces it automatically at placement time; nobody has to manually choose or verify which machine a workload lands on.

KEY POINTS

- Batch, service, and system jobs are first-class scheduler primitives, not workarounds layered onto a services-only platform.
- One cluster can simultaneously run a long-lived web app, a run-once training job, and an always-on inference service.
- Placement constraints — which datacenter, which combination of datacenters, which rack — are declared directly inside the job spec.
- The scheduler enforces placement automatically at deploy time based on the declared constraint.

A batch job pinned to a specific GPU rack

Setting `type = "batch"` marks this job as run-once-and-exit rather than a long-lived service, and the `constraint` block pins it to nodes tagged as belonging to a specific GPU rack — both declared in the same file that also specifies GPU resource needs.

```
job "train-llm" {
  type      = "batch"
  datacenters = ["dc1", "dc2"]

  constraint {
    attribute = "${node.class}"
    value     = "gpu-rack-a"
  }

  group "trainer" {
    count = 1

    task "train" {
      driver = "docker"
      config {
        image = "myorg/trainer:2.1.0"
      }
      resources {
        cpu      = 4000
        memory   = 16384
        device "nvidia/gpu" {
          count = 4
        }
      }
    }
  }
}
```

07 From fragmented ops to unified ops

9:27

Consolidating onto one orchestrator replaces many tools plus siloed, tool-specific knowledge with one tool and shared knowledge across teams.

The 'before' state for an AI/ML organization is fragmented ops: many different tools, and specialized knowledge locked up in whichever team owns each one. The 'after' state, once workloads share one flexible orchestrator, is unified ops: one tool, and shared knowledge that every team can draw on.

The practical effect reaches both ends of the organization. A data scientist can schedule their own training job directly, in minutes, through the same workflow the web team already uses — instead of filing a ticket and waiting days for a dedicated cluster. A DevOps or on-call engineer only has to know one platform deeply; if something breaks at 2 AM, there's one set of logs to check and one mental model to apply, rather than needing to know which of four separate tool chains is the one that failed.

KEY POINTS

- Fragmented ops means multiple tools, each with its own siloed expertise concentrated in one team.
- Unified ops means one tool and one body of knowledge that every team can share.
- Self-service scheduling on a shared orchestrator turns a multi-day wait into a few minutes.
- On-call engineers get one platform and one set of logs to reason about instead of several disconnected tool chains.

Self-service training run

Instead of filing a ticket and waiting for a dedicated cluster, a data scientist submits their own training job to the shared orchestrator directly — the same command-line workflow the web team already uses for deployments — and it starts running within minutes.

```
nomad job run train-job.nomad.hcl
```

08 Staying ready for what's next

10:55

Because AI capabilities keep shifting unpredictably, a flexible orchestrator absorbs new workload types by adding a new job spec, not by rebuilding infrastructure.

AI techniques have moved fast in ways that were hard to predict in advance: transformer models and GPT-scale inference services didn't exist a few years before they became standard infrastructure. A platform that can only run the workload shapes it was originally designed for has to be re-architected every time the underlying techniques shift.

A flexible orchestrator sidesteps that by keeping the infrastructure itself workload-agnostic. New capability shows up as a new declarative job — a new batch job for a new kind of training run, a new service job for a new kind of inference endpoint — submitted to the same scheduler that's already running everything else. The cluster doesn't change; only the specification of what runs on it does. That's the operational simplicity the approach delivers: capability keeps expanding, but the amount of platform work required to support it doesn't scale with it.

KEY POINTS

- AI capabilities have shifted rapidly and unpredictably, e.g. transformer models and GPT-scale inference emerging within a few years.
- A platform limited to its original workload shapes must be rebuilt every time the underlying AI techniques change.
- A flexible orchestrator absorbs a new workload type by adding a new job spec, not by re-architecting the cluster.
- Operational simplicity and growing capability aren't a trade-off when the orchestration layer itself is workload-agnostic.

Rolling out a new capability as a system job

A new GPU telemetry requirement can be met by writing one new job spec of type 'system,' which the scheduler automatically runs as one instance on every eligible node in the cluster — no new cluster, no re-architecture, just a new declarative file submitted to the orchestrator that's already running everything else.

```
job "gpu-metrics-agent" {
  type      = "system"    # one instance per eligible node
  datacenters = ["dc1", "dc2"]

  group "agent" {
    task "exporter" {
      driver = "docker"
      config {
        image = "myorg/gpu-metrics-exporter:1.0.0"
      }
    }
  }
}
```

Glossary

Workload orchestrator

Software that automates deploying, scaling, and healing applications across many servers from a declarative description, instead of requiring manual, per-server operation.

Kubernetes

The most widely used container orchestrator, built primarily to keep long-running containerized services alive and self-healing.

HCL (HashiCorp Configuration Language)

A human-readable declarative configuration language used to write job specs and other infrastructure definitions.

Job spec

A declarative file describing an application's runtime requirements, replica count, resources, and deployment or placement strategy for an orchestrator to execute.

Batch job

A job type meant to run once (or on demand) to completion and then exit, such as a model-training run.

Service job

A job type meant to run continuously and be restarted automatically if it stops, such as an inference API.

System job

A job type that runs one instance on every eligible node in a cluster, commonly used for agents or daemons.

Slurm

An HPC (high-performance computing) job scheduler commonly used in academic and research computing to queue and run GPU or CPU batch jobs.

Airflow

A workflow orchestration tool used to schedule and monitor multi-step data pipelines, typically modeled as directed acyclic graphs (DAGs).

Namespace (Kubernetes)

A way to partition a single cluster into isolated logical sections, often used to separate different teams or workloads sharing one cluster.

Resource quota

A cluster-level limit on how much CPU, memory, or GPU capacity a namespace or team is allowed to consume.

Placement constraint

A rule declared in a job spec that restricts where a workload is allowed to be scheduled, such as a specific datacenter or rack.

Ephemeral workload

A job that runs briefly to complete one task and then terminates, rather than running indefinitely like a long-lived service.

Check yourself

Answer first, then open each one.

Why is it awkward to run a one-off, multi-hour training job as a Kubernetes Deployment rather than a batch-style primitive?

A Deployment is built to keep a fixed number of pods running indefinitely and restart them if they stop; a training run is meant to execute once and then exit. Forcing it into a Deployment fights that core assumption instead of modeling 'run this to completion, then stop,' which is why Kubernetes offers separate Job/CronJob objects for that shape of work.

A company runs one shared Kubernetes cluster for web apps, training, and batch jobs instead of separate clusters. Why does this often create nearly as much operational pain as running three separate clusters?

A single shared cluster avoids the overhead of multiple control planes, but it now needs complex namespace boundaries and resource quotas so teams don't consume each other's capacity. The coordination cost doesn't go away — it just moves from 'managing multiple clusters' to 'policing one crowded cluster.'

Why does bundling replica count, resources, and deployment strategy into one declarative job spec reduce operational burden compared to Kubernetes' Deployment + ConfigMap + Secret + storage pattern?

One artifact describes everything the scheduler needs to place and run the job, so there are fewer files that must stay in sync and fewer places a mismatch between them can hide. A new engineer also only needs to learn one file format instead of learning how several distinct Kubernetes object types relate to one another.

Why does automatic self-healing (detecting that actual state no longer matches desired state, then correcting it) turn a server failure from a crisis into something routine?

The orchestrator continuously compares the declared desired state, such as 3 replicas, against what's actually running, and reschedules missing work automatically. A human no longer has to notice the failure, log in, and manually restart anything — the corrective loop runs on its own instead of depending on someone being paged.

If four teams each adopt the tool that best fits their immediate workload (Kubernetes, Slurm, Airflow, ad hoc scripts), what cost does the organization pay later that isn't visible at adoption time?

When something breaks, there's no shared toolset or mental model to diagnose it with; every incident requires finding whoever knows that specific tool. Troubleshooting time and on-call complexity end up scaling with the number of tools in use, not with the number of failures that actually occur.

Why does treating a new AI capability as 'a new job spec' rather than 'new infrastructure' deliver operational simplicity without sacrificing capability?

If every new kind of workload required new clusters or new tooling, the platform would need to be rebuilt each time AI techniques changed, as they did with transformer models and GPT-scale inference. Describing the new workload as another declarative job on the existing scheduler keeps the infrastructure itself unchanged — only the specification of what runs on it grows.

Where to go next

- Read the HashiCorp Nomad documentation on job types (service, batch, system) and compare its scheduler model to Kubernetes' Deployment, Job, and CronJob objects.

- Set up a small local workload orchestrator cluster and submit a batch job alongside a long-running service job to see both scheduled from the same job-file format.
 - Compare Slurm's queue-based scheduling model to a Kubernetes Job object for GPU-bound training workloads, noting what each is optimized for.
 - Read up on Kubernetes Namespace and ResourceQuota objects to see the multi-tenant controls needed when several teams share one cluster.
-

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.